UNIVERSITY OF BRISTOL

School of Mathematics

Algorithms and Programming in C(++) and R
MATH10017
(Paper code MATH-10017W)

Winter 2026 2 hour 30 minutes

This paper contains two sections: Section A and Section B.
Each section should be answered in a separate booklet.

Section A contains FIVE questions and Section B contains TWO questions.
All SEVEN answers will be used for assessment.

Calculators are not permitted.

MOCK EXAM 2
SOLUTIONS

Do not turn over until instructed.

Section A: Short Questions

A1. (**8 marks**)

(a) (**2 marks**) Which one of the following statements is **wrong**?

(A) In C, the name of an array can be used as a pointer to its first element.

(B) In C, memory obtained with `malloc` is taken from the heap.

(C) In C, the local variables of a function live on the stack.

(D) In C, the stack can never run out of space, so a program can recurse as deeply as it likes.

(E) In R, all the elements of a vector have the same type.

Solution

[D is wrong. The stack has a limited size, so recursion that is too deep (for example one that never reaches its base case) causes a stack overflow.]

(b) (**3 marks**) Explain the difference between **stack** and **heap** memory in C. State one advantage of each.

Solution

[In the call-stack model, stack memory stores a function call's parameters and local variables and is released automatically when the call returns. The stack has limited space. Heap memory is requested with `malloc` while the program runs and remains allocated after the allocating function returns, until it is released with `free`. Advantage of the stack: automatic allocation and release. Advantage of the heap: an allocation can outlive the function that created it and can typically be much larger than a local array.]

(c) (**3 marks**) What is printed by this R code? Briefly justify your answer.

```
x <- seq(2, 10, by = 2)
print(x[x > 5])
```

Solution

[`seq(2, 10, by = 2)` is the vector `2 4 6 8 10`. The condition `x > 5` selects the elements 6, 8 and 10, so the code prints]

```
[1] 6 8 10
```

A2. (**8 marks**)

Read the following C program and answer the questions.

```
#include <stdio.h>

int mystery(int n){
    int r = 0;
    for(int i = 1; i <= n; i++){
        r = r + i;
    }
    return r;
}

int main(void){
    printf("%d\n", mystery(4));
}
```

(a) (**3 marks**) What is the printed output of the program?

Solution

[It prints 10, because `mystery(4)` computes $1 + 2 + 3 + 4 = 10$.]

(b) (**3 marks**) For a positive integer `n`, what quantity does `mystery(n)` compute?

Solution

[It computes the sum of the whole numbers from 1 to `n`, that is $1 + 2 + \cdots + n$.]

(c) (**2 marks**) Change the program so that `mystery(n)` instead computes the **product** $1 \times 2 \times \cdots \times n$. State only the lines you have changed.

Solution

[Start the accumulator at 1 (not 0) and multiply:]

```
int r = 1;
r = r * i;
```

A3. (**8 marks**)

The following program uses a one-dimensional C array to store a 3 by 3 matrix in **column-major** order. Indices start at zero.

```
#include <stdio.h>

int elem(int M[], int i, int j){
    /* YOUR CODE HERE */
}

int col_total(int M[], int j){
    int total = 0;
    for(int i = 0; i < 3; i++){
        /* YOUR CODE HERE */
    }
    return total;
}

int main(void){
    int M[] = {1,2,3, 4,5,6, 7,8,9};
    printf("%d\n", elem(M, 0, 2));
    printf("%d\n", col_total(M, 1));
}
```

(a) (**2 marks**) Complete `elem` using one line of code, so that it returns the (i, j) entry of a 3 by 3 matrix stored in **column-major** order.

Solution

```
int elem(int M[], int i, int j){
    return M[j*3 + i];
}
```

(b) (**4 marks**) Complete the body of the loop in `col_total`, which should return the sum of column `j`, using one line of code. Then state the two numbers printed by the completed program.

Solution

[The missing line is]

```
total = total + elem(M, i, j);
```

[In column-major order the matrix is]

$$\begin{pmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{pmatrix},$$

[so `elem(M, 0, 2)` is 7 and `col_total(M, 1)` is $4+5+6=15$. The program prints]

```
7
15
```

(c) (**2 marks**) Suppose the matrix instead has `m` rows and the loop bound is changed accordingly. How many additions does `col_total` perform, and what is its time complexity in $O(\cdot)$ notation?

Solution

[The loop runs m times, so m additions and a time complexity of $O(m)$.]

A4. (**8 marks**)

Answer the following questions in R.

(a) (**3 marks**) What is printed by this code?

```
v <- c(3, 8, 1, 6)
print(v[c(TRUE, FALSE, TRUE, FALSE)])
```

Solution

[A logical vector selects the elements where it is `TRUE`, here positions 1 and 3, so the code prints]

```
[1] 3 1
```

(b) (**3 marks**) The function below should return the sum of the **positive** elements of `v`. Assume that `v` contains at least one element. Complete the two missing expressions.

```
sum_pos <- function(v){
  s <- 0
  for(ii in 1:length(v)){
    if(____________________){
      ____________________
    }
  }
  return(s)
}
```

Solution

```
sum_pos <- function(v){
  s <- 0
  for(ii in 1:length(v)){
    if(v[ii] > 0){
      s <- s + v[ii]
    }
  }
  return(s)
}
```

(c) (**2 marks**) Write a vectorised version of `sum_pos` using a single line in the function body.

Solution

```
sum_pos_v <- function(v){
  sum(v[v > 0])
}
```

A5. (**8 marks**)

Consider the following R matrix.

```
X <- matrix(1:6, nrow = 2)
```

(a) (**3 marks**) Write down the first row `X[1, ]` and the second column `X[ , 2]`.

Solution

[R fills a matrix column by column by default, so]

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

[Hence `X[1, ]` is `1 3 5` and `X[ , 2]` is `3 4`.]

(b) (**3 marks**) Complete the function below so that it returns the largest value in each **column** of `X`. Also state the value returned when it is called on `X`.

```
col_max <- function(X){
  ----------------------
}
```

Solution

```
col_max <- function(X){
  apply(X, 2, max)
}
```

[The margin is 2 because the function is applied to columns. On `X` it returns `2 4 6`.]

(c) (**2 marks**) What would `apply(X, 1, max)` return instead, and why is it different?

Solution

[It returns `5 6`. The margin 1 applies the function to the **rows** rather than the columns, so it gives the largest value in each of the two rows.]

Section B: Longer Questions

Please use a new booklet for this Part.

B1. (**30 marks**)

A weather service records the midday temperature (in whole degrees Celsius) at three stations over five days. The entry $T_{i,j}$ is the temperature at station i on day j. The matrix is stored in a one-dimensional C array in **row-major** order. Use zero-based indexing throughout this question.

For example, the original readings are:

```
int T[] = {18,20,21,19,17,
           22,23,22,24,25,
           15,14,16,15,13};
```

Treat each part independently, starting from the original readings. You only need to write the requested functions. You may use functions from earlier parts of this question even if you did not answer those parts.

(a) (**10 marks**) Write a function `void swap_days(int T[], int j, int k)` that swaps the readings for days `j` and `k` at every station. Modify `T` itself. You may assume that both day indices are between 0 and 4 inclusive.

Solution

```
void swap_days(int T[], int j, int k){
    for(int i = 0; i < 3; i++){
        int temporary = T[i*5 + j];
        T[i*5 + j] = T[i*5 + k];
        T[i*5 + k] = temporary;
    }
}
```

[The loop visits the three stations. A temporary variable preserves one reading while the other is copied. For example, `swap_days(T, 0, 4)` changes the first station's readings to `17, 20, 21, 19, 18`. If `j == k`, the readings are unchanged.]

(b) (**10 marks**) Write a function `void reverse_days(int T[])` that reverses the order of the five days at every station: day 0 becomes day 4, day 1 becomes day 3, and so on. Modify `T` itself. Your solution must use a loop and call `swap_days` from part (a).

Solution

```
void reverse_days(int T[]){
    for(int j = 0; j < 5/2; j++){
        swap_days(T, j, 4-j);
    }
}
```

[Integer division makes `5/2` equal to 2. The loop swaps days 0 and 4, then days 1 and 3, leaving the middle day unchanged. Iterating over all five days would undo the swaps. The first station's reversed readings are `17, 19, 21, 20, 18`.]

(c) (**10 marks**) Write a function `void transpose(const int T[], int out[])` that stores the transpose of `T` in `out`, leaving `T` unchanged. The result has five rows and three columns and must also be stored in **row-major** order. Its entry at row `j`, column `i` must equal the original entry $T_{i,j}$. The caller supplies `out` as a separate array with space for 15 integers; you do not need to allocate memory.

Solution

```
void transpose(const int T[], int out[]){
    for(int i = 0; i < 3; i++){
        for(int j = 0; j < 5; j++){
            out[j*3 + i] = T[i*5 + j];
        }
    }
}
```

[The input index uses five columns, whereas the output index uses three. For the original readings, the transposed matrix is:]

```
18 22 15
20 23 14
21 22 16
19 24 15
17 25 13
```

B2. (**30 marks**)

An R data frame stores information about the products sold in a shop: their name, unit price, and the number of units currently in stock.

```
products <- data.frame(
  name  = c("pen", "pad", "clip", "tape"),
  price = c(4, 5, 2, 3),
  stock = c(20, 16, 30, 25)
)
```

Treat each part independently, starting from the original data frame shown above.

You may use functions from earlier parts of this question even if you did not answer those parts.

(a) (**10 marks**) Using the data frame above, write one R expression for each of the following:

(i) return the whole `price` column;

(ii) return the stock of `"clip"` (the third product);

(iii) return the names of the products whose price is at least 3; and

(iv) change the stock of `"clip"` to 90.

Solution

```
products$price                          # (i)
products$stock[3]                       # (ii)
products$name[products$price >= 3]      # (iii)
products$stock[3] <- 90                 # (iv)
```

[For (ii), `products[3, 3]` is also correct. For (iv), `products[3, 3] <- 90` is also correct.]

(b) (**10 marks**) In both parts below, `df` has the same columns as `products`. Your functions must not use an explicit `for` or `while` loop.

(i) (**5 marks**) Write a function `find_products(df, p, s)` that returns the names of products whose price is at most `p` and whose stock is less than `s`. Both conditions must hold.

Solution

```
find_products <- function(df, p, s){
  return(df$name[df$price <= p & df$stock < s])
}
```

[For example, `find_products(products, 4, 26)` returns `c("pen", "tape")`. Both conditions must be checked element by element using `&`, rather than `&&`.]

(ii) (**5 marks**) Write a function `restock(df, s, amount)` that increases the stock of every product currently holding fewer than `s` units by `amount`. Stocks of other products must remain unchanged. Return the updated data frame.

Solution

```
restock <- function(df, s, amount){
  selected <- df$stock < s
  df$stock[selected] <- df$stock[selected] + amount
  return(df)
}
```

[For example, `restock(products, 25, 10)` returns a data frame whose stock column is `c(30, 26, 30, 25)`. A stock equal to the threshold is left unchanged.]

(c) (**10 marks**) Write an R function `most_valuable(df)` that returns the names of **all** products whose value is the largest. The value of a product is defined as its `price` times its `stock`. Your function must also work when several products tie for the largest value.

Solution

```
most_valuable <- function(df){
  values <- df$price * df$stock
  best <- max(values)
  return(df$name[values == best])
}
```

[For the data frame given in the question, the values are 80, 80, 60 and 75, so the function returns `c("pen", "pad")`. Using logical indexing with `==` ensures that every tied product is returned.]